FACE Prep

System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the

Select Language : C (GCC 9.2.0)

```c
#include<stdio.h>
int main(){
  int p,r;
  int i,j,k;
  printf("Enter number of processes: \n");
  scanf("%d",&p);
  printf("Enter number of resource types: \n");
  scanf("%d",&r);
  int alloc[p][r];
  int max[p][r];
  int need[p][r];
  int ava[r];
  int work[r];
  int finish[p];
  int safe[p];
  printf("Enter Allocation Matrix:\n");
  for( i=0;i<p;i++){
    for(j=0;j<r;j++){
      scanf("%d",&alloc[i][j]);
    }
  }
  printf("Enter Max Matrix:\n");
  for(i=0;i<p;i++){
    for(j=0;j<r;j++){
      scanf("%d",&max[i][j]);
    }
  }
  printf("Enter Available Resources: \n");
```

Test Cases | Run Sample Cases | Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2